

Introduction à la Programmation MPI
Contrôle continu du 15/11/2024 – Ce document contient 4 pages
1h30 – Tout document manuscrit autorisé (support informatique interdit)

Exercice I : Petit programme MPI

Soit la section de code suivante exécutée par tous les processus d'un programme MPI (il y'a au moins 2 processus, on suppose le programme déjà initialisé avec MPI_Init) :

```
static int val ;  
int rang;  
  
MPI_Comm_rank(MPI_COMM_WORLD, &rang) ;  
  
val = 1;  
if (rang == 0)  
    val *= 10;  
  
MPI_Barrier(MPI_COMM_WORLD) ;  
  
val *= 2 ;  
if (rang == 1)  
    val *= 100 ;  
  
MPI_Barrier(MPI_COMM_WORLD) ;
```

Question 1 :

Pourrait-on supprimer les barrières MPI sans influencer le résultat du programme ? (justifiez brièvement)

Question 2 :

Suite à l'exécution de ces lignes, quelle est la valeur de la variable `val` en fonction du rang MPI ? (justifiez brièvement)

Exercice II : Quelle est la bonne valeur ?

Soit la section de code suivante exécutée par tous les processus d'un programme MPI (il y'a au moins 3 processus) :

```
int rang, val, valA, valB ; MPI_Status sta;  
  
MPI_Comm_rank(MPI_COMM_WORLD, &rang) ;  
  
if (rang == 0) {  
    MPI_Recv(&valA, 1, MPI_INT, MPI_ANY_SOURCE, 1000, MPI_COMM_WORLD, &sta) ;  
    MPI_Recv(&valB, 1, MPI_INT, MPI_ANY_SOURCE, 1111, MPI_COMM_WORLD, &sta) ;  
} else if (rang == 1) {  
    val = rang ;  
    MPI_Send(&val, 1, MPI_INT, 0, 1111, MPI_COMM_WORLD) ;  
} else if (rang == 2) {  
    val = rang ;  
    MPI_Send(&val, 1, MPI_INT, 0, 1000, MPI_COMM_WORLD) ;  
}
```

Question : Suite à l'exécution de ces lignes, quelles sont les valeurs des variables valA et valB du processus de rang 0 (justifiez) ?

Exercice III : Correct/Pas correct

Pour chacune des sections de code suivantes (numérotées de a à d), dites :

- si elle fait l'objet d'un risque de blocage (« deadlock ») ;
- ou bien si les contenus des messages risquent d'être erronés (« erroné ») ;
- ou bien si la section est sûre (« correct »).

Justifiez brièvement vos choix.

Dans le cas où une section de code n'est pas correcte, apportez une correction pour que la section soit sûre.

Dans tout ce qui suit, on désigne par rang le rang du processus MPI, et par P la taille du communicateur MPI_COMM_WORLD. On suppose que le programme a correctement initialisé MPI.

a) Chaque processus doit afficher les valeurs collectées des autres processus

```
int arr[P];
MPI_Gather(&rang,1,MPI_INT, arr, P, MPI_INT, 0, MPI_COMM_WORLD);
for(int i=0 ; i<P; i++)
    printf("P%d : arr[%d]=%d\n", rang, i, arr[i]);
```

b) Chaque processus échange vals[0] et vals[2] avec les processus rang-1 et rang+1 s'ils existent (les valeurs sont respectivement reçues dans vals[1] et vals[3])

```
int vals[4] = {0,0,0,0};
if (rang>0) {
    vals[0]=1;
    MPI_Send(vals+0,1,MPI_INT, rang-1, 1000,MPI_COMM_WORLD);
    MPI_Recv(vals+1,1,MPI_INT, rang-1, 1000,MPI_COMM_WORLD, MPI_STATUS_IGNORE);
}
if (rang<(P-1)) {
    vals[2]=2;
    MPI_Send(vals+2,1,MPI_INT, rang+1, 1000,MPI_COMM_WORLD);
    MPI_Recv(vals+3,1,MPI_INT, rang+1, 1000,MPI_COMM_WORLD, MPI_STATUS_IGNORE);
}
printf("P%d vals=%d,%d\n", rang, vals[1],vals[3]);
```

c) Le processus 0 veut envoyer le message m = 100 au processus 1

```
if (rang == 0) {
    m = 100 ;
    MPI_Send(&m, 1, MPI_INT, 1, 1000, MPI_COMM_WORLD) ;
} else if (rang == 1) {
    MPI_Request req ;
    MPI_Irecv(&m, 1, MPI_INT, 0, 1000, MPI_COMM_WORLD, &req) ;
    MPI_Wait(&req, MPI_STATUS_IGNORE) ;
}
```

d) Le processus 0 diffuse une valeur aux autres processus.

```
int som ;
if (rang == 0) {
    som = 1 ;
    MPI_Bcast(&som,1,MPI_INT, 0, MPI_COMM_WORLD) ;
} else {
    MPI_Recv(&som,1,MPI_INT, 0, 1000, MPI_COMM_WORLD, MPI_STATUS_IGNORE) ;
}
```

Exercice IV : Implémentation MPI d'un scan

Listing de la fonction `scan_som` et son programme principal : complétez-le.

```
1. #include <mpi.h>
2. int scan_som(int in) {
3.     int rang, P, out;
4.     MPI_Status sta;
5.     /* A COMPLETER : il faut affecter la variable rang */
6.     /* A COMPLETER : il faut affecter la variable P */
7.     if (rang != 0)
8.         /* A COMPLETER */
9.     else
10.        out = 0 ;
11.    out = out + in;
12.    if (rang != (P-1))
13.        /* A COMPLETER */
14.    return out;
15. }
16. int main(int argc, char **argv) {
17.     int rang, P, res;
18.     /* A COMPLETER */
19.     /* A COMPLETER : le rang doit être affecté */
20.     res = scan_som(rang);
21.     printf("P%d, resultat scan = %d\n", rang, res);
22.     /* A COMPLETER */
23.     return 0;
24. }
```

Le « scan somme » est une communication collective qui prend en argument, sur le processus MPI de rang rang , un entier in_{rang} , et qui retourne un entier out_{rang} :

$$\text{out}_{\text{rang}} = \sum_{0 \leq i \leq \text{rang}} \text{in}_i = \text{in}_0 + \text{in}_1 + \dots + \text{in}_p + \dots + \text{in}_{\text{rang}}$$

En particulier, $\text{out}_0 = \text{in}_0$ et $\text{out}_{P-1} = \sum_{0 \leq i < P} \text{in}_i$, où P est le nombre total de processus MPI.

Soit la fonction

`int scan_som(int in)`

qui prend en argument in_{rang} et qui retourne out_{rang} sur le processus de rang rang .

Rang MPI	0	1	2
in	6	3	5
res	6	9	14

Exemple des valeurs (in , res) lors d'un appel à `res = scan_som(in)` pour 3 processus MPI

La fonction `scan_som` est implémentée dans le listing ci-dessus, ainsi qu'un programme principal qui

utilise cette fonction scan_som. Cependant le listing n'est pas complet.

Travail à faire : compléter les lignes /* A COMPLETER */ pour que la fonction scan_som et le programme principal soient corrects.

Exercice V : Implémentation d'une réduction/distribution

Soit la fonction :

```
void reduction_distribution_som( int *sendbuf, int *recvbuf, int recvcount ) ;
```

avec

- sendbuf (IN) : tableau d'entiers pré-alloué à la taille $P * \text{recvcount}$
- recvbuf (OUT) : tableau d'entiers pré-alloué à la taille recvcount
- recvcount (IN) : nombre d'éléments à recevoir par processus (tous les processus appellent la fonction avec la même valeur de recvcount)

Cette fonction est une opération collective : appelée par tous les processus MPI, elle effectue :

1. une réduction somme sur tous les éléments du vecteur sendbuf,
2. puis découpe le résultat en P sous-vecteurs de taille recvcount ,
3. et distribue le vecteur réduit sur tous les processus : les recvcount premiers éléments du résultat vont au premier processus, les recvcount suivants vont au deuxième processus, etc... jusqu'au derniers recvcount éléments qui vont au dernier processus.

Exemple avec 3 processus MPI		
données P0	données P1	données P2
Entrées : recvcount = 2 sendbuf = [1, 3, 4, 0, 6, 5] Sortie : recvbuf = [5, 8]	Entrées : recvcount = 2 sendbuf = [0, 2, 2, 1, 1, 0] Sortie : recvbuf = [6, 5]	Entrées : recvcount = 2 sendbuf = [4, 3, 0, 4, 2, 1] Sortie : recvbuf = [9, 6]

Question :

Sans utiliser la fonction MPI_Reduce_scatter_block ni MPI_Reduce_scatter, proposez une implémentation de la fonction reduction_distribution_som en utilisant uniquement des communications collectives avec racine.